WOXSEN UNIVERSITY | SCHOOL OF TECHNOLOGY | DBMS COURSEWORK

Railway Ticket Reservation and Journey Management System

REVIEW 2: Normalization, DDL Constraints, and SQL Query Implementation

Student Name: Heman | Roll Number: 25WU0102097 | Section: AIML Rhinos
Faculty Mentor: Prof. Sahithi | School of Technology, Woxsen University | Week 11

Refined and Implemented Relational Schema

14 Production-Ready Relational Tables

- **STATIONS:** PK: station_id | UNIQUE: station_code — Physical railway stations
- **ROUTES:** PK: route_id | UNIQUE: route_code | FK: source_stn, dest_stn — Track corridors and total distance
- **TRAIN_STOPS:** PK: stop_id | FK: route_id, station_id | UNIQUE: (route_id, stop_seq) — Ordered station stops and arrival/departure times
- **TRAINS:** PK: train_id | UNIQUE: train_number | FK: route_id — Locomotive catalog (Vande Bharat, Rajdhani)
- **TRAIN_RUNS:** PK: run_id | FK: train_id | UNIQUE: (train_id, run_date) — Operational calendar dates decoupled from trains
- **COACHES & SEATS:** PK: coach_id, seat_id | UNIQUE: (coach_id, seat_number) — Physical carriages and seat berth units
- **PASSENGERS:** PK: passenger_id | UNIQUE: email | CHECK: age BETWEEN 1 AND 119 — Decoupled traveler identity records
- **BOOKINGS:** PK: booking_id | UNIQUE: pnr_number | FK: run_id, passenger_id — Transactional booking records with fare and status
- **JOURNEY_SEGMENTS:** PK: segment_id | FK: booking_id, seat_id, run_id — Half-open seat intervals [start_seq, end_seq)
- **WAITLISTS:** PK: waitlist_id | FK: booking_id, run_id — FIFO queue positions for exhausted segments
- **PAYMENTS & CANCELS:** PK: payment_id, cancellation_id | UNIQUE: txn_ref, booking_id — Audited payment and refund ledgers

Formal Functional Dependency (FD) Analysis

Minimal Candidate Keys and Functional Dependencies

✔ **stations: CK = {station_id}, {station_code}:**
station_id -> {station_code, station_name, city, state}
station_code -> {station_id, station_name, city, state}

✔ **trains: CK = {train_id}, {train_number}:**
train_id -> {train_number, train_name, train_type, route_id}
train_number -> {train_id, train_name, train_type, route_id}

✔ **train_runs: CK = {run_id}, {train_id, run_date}:**
run_id -> {train_id, run_date, status}
{train_id, run_date} -> {run_id, status}

✔ **train_stops: CK = {stop_id}, {route_id, stop_seq}:**
stop_id -> {route_id, station_id, stop_seq, distance, arrival, dep}
{route_id, stop_seq} -> {stop_id, station_id, distance, arrival, dep}

✔ **passengers: CK = {passenger_id}, {email}:**
passenger_id -> {full_name, gender, age, email, phone, id_card_type, id_card_num}

✔ **bookings: CK = {booking_id}, {pnr_number}:**
pnr_number -> {booking_id, run_id, passenger_id, orig_stn, dest_stn, status, fare}

Step-by-Step Normalization to Third Normal Form

1NF: Atomic Attributes

Problem in UNF:
Spreadsheet contains repeating groups and multivalued stop lists (e.g. 'NDLS, CNB, PRYJ').

Decomposition:
- Decompose stops into discrete tuples: train_stops(stop_id, route_id, station_id, stop_sequence).
- Remove multivalued passenger lists.
- Every cell contains strictly atomic scalar values.

2NF: No Partial Dependencies

Problem in 1NF:
In composite key {train_id, run_date}, train_name depends solely on train_id, violating 2NF.

Decomposition:
- Isolate trains(train_id, train_name, train_type) from dated train_runs(run_id, train_id, run_date).
- Isolate coaches and seats.
- Isolate passenger profile attributes from bookings.

3NF: No Transitive Dependencies

Problem in 2NF:
In bookings, booking_id -> cancellation_id -> {refund_amt, fee}. Non-prime attributes depend transitively on cancellation_id.

Decomposition:
- Extract cancellations into dedicated table.
- Extract payments into dedicated table.
- Extract fare_rules into separate pricing catalog.

Relational Data Dictionary Summary

Core Entity Column Constraints & Types

Table	Column	Type	Null?	Key / Constraints	Description
stations	station_code	VARCHAR(10)	No	UNIQUE, INDEX	IRCTC station identifier (e.g. NDLS)
train_stops	stop_sequence	INT	No	CHECK (> 0), UQ(route, seq)	Ordered sequence position
train_runs	run_date	DATE	No	UQ(train_id, run_date)	Scheduled operational date
passengers	age	INT	No	CHECK (age > 0 AND age < 120)	Passenger legal age
bookings	total_fare	DECIMAL(10,2)	No	CHECK (total_fare >= 0)	Exact ticket purchase price
journey_segments	allocation_status	VARCHAR(20)	No	DEFAULT 'ACTIVE', INDEX	'ACTIVE' or 'CANCELLED'
cancellations	refund_amount	DECIMAL(10,2)	No	CHECK (refund_amount >= 0)	Refund processed to passenger

MySQL DDL and Declarative Integrity Constraints

Production MySQL 8.x DDL Code Extracts

```
CREATE TABLE journey_segments (
    segment_id INT AUTO_INCREMENT PRIMARY KEY,
    booking_id INT NOT NULL,
    seat_id INT NOT NULL,
    run_id INT NOT NULL,
    start_stop_seq INT NOT NULL,
    end_stop_seq INT NOT NULL,
    allocation_status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
    CONSTRAINT fk_seg_booking FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE CASCADE,
    CONSTRAINT fk_seg_seat FOREIGN KEY (seat_id) REFERENCES seats(seat_id) ON DELETE RESTRICT,
    CONSTRAINT chk_seg_seq_order CHECK (end_stop_seq > start_stop_seq),
    INDEX idx_seat_run_alloc (run_id, seat_id, allocation_status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE waitlists (
    waitlist_id INT AUTO_INCREMENT PRIMARY KEY,
    booking_id INT NOT NULL,
    run_id INT NOT NULL,
    travel_class VARCHAR(10) NOT NULL,
    position INT NOT NULL,
    waitlist_status VARCHAR(20) NOT NULL DEFAULT 'WAITING',
    CONSTRAINT chk_wl_pos_positive CHECK (position > 0),
    INDEX idx_wl_queue (run_id, travel_class, position)
) ENGINE=InnoDB;
```

Segment-Wise Seat Availability and Overlap Algorithm

The Half-Open Interval Formula [s, e)

Interval Overlap Condition:
For existing allocation [s, e) and requested interval [r_start, r_end):

OVERLAP = (s < r_end) AND (r_start < e)

Relational SQL Implementation:

```
SELECT seat_id FROM journey_segments
WHERE run_id = :run_id
  AND allocation_status = 'ACTIVE'
  AND start_stop_seq < :req_end
  AND :req_start < end_stop_seq;
```

Any seat returned by this query is currently occupied over the requested segment.

Empirical Seat Reuse Demonstration

Route: NDLS(1) -> CNB(2) -> PRYJ(3) -> BSB(4)

Case 1: Valid Non-Overlapping Reuse (Seat C1-1)
- Passenger 1 books NDLS -> CNB: [1, 2)
- Passenger 2 books CNB -> BSB: [2, 4)

Overlap Test: 1 < 4 (True) AND 2 < 2 (False) => NO CONFLICT!
Both confirmed simultaneously on Seat C1-1!

Case 2: Conflicting Request Blocked
- Passenger 3 requests NDLS -> BSB: [1, 4)

Overlap Test with [1, 2): 1 < 4 (True) AND 1 < 2 (True) => CONFLICT!
Seat C1-1 is blocked.

Waitlist Queue & Atomic Promotion Workflow

Deterministic Cancellation & Promotion Pipeline

✔ **1. Queue Serialization:** When a travel class is exhausted for an interval, bookings are placed into waitlists with position = MAX(position) + 1. Scoped strictly by (run_id, travel_class).

✔ **2. Cancellation & Deterministic Refund:** Cancellation of confirmed ticket applies a 20% fee deduction (retained) and 80% refund. The journey_segments record transitions to CANCELLED.

✔ **3. Re-verification of Waitlist Candidates:** Waiting entries are retrieved in FIFO order (position ASC). The candidate's exact [origin, dest) interval is re-checked against all coach seats.

✔ **4. Atomic Promotion Execution:** If an eligible seat is found, a new JourneySegment is created, candidate booking becomes CONFIRMED, and waitlist status becomes PROMOTED within the same database transaction.

SQL Implementation: Joins, Nested Queries & Views

Executed SQL Query Extracts

```
-- 1. COACH OCCUPANCY WITH MULTI-TABLE JOINS & AGGREGATION
SELECT c.coach_code, c.travel_class, COUNT(DISTINCT js.segment_id) AS occupied,
       ROUND(COUNT(DISTINCT js.segment_id) * 100.0 / c.total_seats, 2) AS occupancy_pct
FROM coaches c JOIN seats s ON c.coach_id = s.coach_id
LEFT JOIN journey_segments js ON s.seat_id = js.seat_id AND js.allocation_status = 'ACTIVE'
GROUP BY c.coach_id, c.coach_code, c.travel_class, c.total_seats;

-- 2. FINANCIAL RECONCILIATION AUDIT (SCALAR NESTED SUBQUERIES)
SELECT
    (SELECT SUM(total_fare) FROM bookings) AS gross_booked_fare,
    (SELECT SUM(amount) FROM payments WHERE payment_status = 'COMPLETED') AS net_collected,
    (SELECT SUM(refund_amount) FROM cancellations) AS total_refunded;
```

12 Executed Benchmark SQL Reports

Comprehensive SQL Reporting Catalog

- 1. Segment-Wise Seat Availability by Train & Class
- 2. Passenger Booking History & Detailed Itinerary
- 3. Active Waitlist Queue Priority & Position
- 4. Coach Occupancy Percentage Ratio
- 5. Route Demand & Passenger Volume Analysis
- 6. Cancellation Counts, Fees & Refund Disbursed
- 7. Complete Journey Timetable & Station Stops
- 8. Revenue Grouped by Train and Travel Class
- 9. Financial Reconciliation Audit (Fares vs Refunds)
- 10. Status Distribution (Confirmed vs WL vs Cancel)
- 11. Train Stop Timetable & Distance Analysis
- 12. High-Value Frequent Passengers (Nested Query)

Database Prototype and Automated Test Evidence

Actual Execution Evidence (16 Passed Tests in 0.36s)

Automated Test Suite Output (pytest 8.4.2):

```
Railway_Reservation_System/tests/test_availability.py::test_interval_overlap_logic PASSED [  6%]
Railway_Reservation_System/tests/test_availability.py::test_non_overlapping_seat_reuse PASSED [ 12%]
Railway_Reservation_System/tests/test_availability.py::test_conflicting_interval_detection PASSED [ 18%]
Railway_Reservation_System/tests/test_cancellations.py::test_eligible_cancellation_and_refund_policy PASSED [ 31%]
Railway_Reservation_System/tests/test_cancellations.py::test_automated_waitlist_promotion PASSED [ 43%]
Railway_Reservation_System/tests/test_integrity.py::test_unique_train_number_constraint PASSED [ 50%]
Railway_Reservation_System/tests/test_integrity.py::test_passenger_age_check_constraint PASSED [ 62%]
Railway_Reservation_System/tests/test_reports.py::test_all_12_reports_execute_successfully PASSED [ 81%]
Railway_Reservation_System/tests/test_reservations.py::test_valid_booking_success PASSED [ 87%]
============================== 16 passed in 0.36s ==============================
```

Milestone Review 3 Objectives & Roadmap

Final Review 3 Deliverable Goals

🎯 **Complete End-to-End Browser UI:** Deploy intuitive dashboard, interactive train search, booking wizard, and ERS ticket slip views.

🎯 **Concurrency & Stress Verification:** Verify transaction boundaries and row locking under multi-user simulated requests.

🎯 **Comprehensive Academic Project Report:** Compile full 23-section project report with relational schema, data dictionary, and viva prep.

🎯 **Live Demonstration Sequence:** Deliver repeatable 5-8 minute live demonstration script showcasing seat reuse and waitlist promotion.